

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Текст программы (индивидуальная часть)

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 12 03-1-ЛУ

Исполнители:

Студент группы БДРИП242

_____ / А. А. Клушин /

«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.03-04 12 03-1-ЛЮ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Текст программы (индивидуальная часть)

RU.17701729.09.03-04 12 03-1

Листов 13

Москва 2026

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

В настоящем документе приведены сведения о составе и структуре серверной части программы «Платформа для no-code автоматизации бизнес-процессов». Серверная часть написана на языке Kotlin с использованием фреймворка Spring Boot.

Настоящий документ разработан в соответствии с ГОСТ 19.401-78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Краткая характеристика области применения	4
2. ТЕКСТ ПРОГРАММЫ	5
2.1. Размещение исходного кода и развёрнутая инстанция	5
2.2. Структура директории серверной части	5
2.3. Модули серверной части	6
2.4. Ключевые классы	7
2.5. Технологические зависимости	9
2.6. Инструкция по сборке	10
2.7. Сборка Docker-образа	10
2.8. Запуск тестов	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

«Платформа для no-code автоматизации бизнес-процессов» — серверная часть (внутреннее имя продукта команды — FluxPilot, серверная часть). Настоящий документ описывает текст программы индивидуальной части курсового проекта студента Клушина А.А.

1.2. Краткая характеристика области применения

Серверная часть обеспечивает: REST API управления workflow с автоматической документацией OpenAPI 3.0; движок параллельного исполнения графов workflow с топологической сортировкой по алгоритму Кана и event-driven исполнением через CompletableFuture; изолированную Docker-песочницу для пользовательского Python- и JavaScript-кода; миграции схемы PostgreSQL через Liquibase; контент-адресуемое хранение конфигов в MinIO с дедупликацией по SHA-256; систему триггеров трёх типов (manual, scheduler, webhook); WebSocket-трансляцию событий жизненного цикла исполнения через STOMP поверх SockJS.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ТЕКСТ ПРОГРАММЫ

2.1. Размещение исходного кода и развёрнутая инстанция

Полный текст серверной части размещён в открытом Git-репозитории проекта по адресу:

github.com/arklual/cursach

Серверная часть расположена в директории backend/. Снимок исходного кода, соответствующий настоящему документу, зафиксирован в репозитории тегом v1.0-defense (создаётся непосредственно перед защитой). Репозиторий публичен и не требует аутентификации для чтения.

Серверная часть развёрнута в продакшен-окружении и обслуживает запросы публичной инстанции по адресу <https://fluxpilot.ru/>. REST API доступен по префиксу /v1; интерактивная документация Swagger UI — по адресу <https://fluxpilot.ru/v1/swagger-ui.html>. Развёртывание выполнено по конфигурации deploy/docker-compose.prod.yml (Spring Boot backend за Nginx, PostgreSQL 16, MinIO, docker-socket-proxy).

2.2. Структура директории серверной части

Путь	Назначение
src/main/kotlin/	Исходный код серверной части (одиннадцать пакетов, около 55 файлов .kt)
src/main/resources/	Конфигурация Spring (application.properties, application.yml), миграции Liquibase (db/changelog/)
src/test/kotlin/	Автоматические тесты (модульные на JUnit 5, интеграционные на Spring Boot Test + Testcontainers)
pom.xml	Maven-конфигурация: зависимости, плагины сборки и тестирования, jacoco-maven-plugin для отчёта покрытия
Dockerfile	Описание сборки production Docker-образа на базе eclipse-temurin:24-jre
.mvn/	Maven Wrapper (для воспроизводимой сборки без глобально установленного Maven)
target/	Артефакты сборки (создаётся при mvn package): fat-JAR, surefire-отчёты, JaCoCo-отчёт

Таблица 1. Структура директории backend/

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2.3. Модули серверной части

Все исходные коды серверной части размещены в корневом пакете `ru.startem.aelevena`, разделённом на одиннадцать модулей-пакетов с однонаправленными зависимостями.

Пакет	Назначение
api	REST-контроллеры: WorkflowsController, RunsController, WorkflowVersionsController, WorkflowSnapshotsController, TriggersController, AbAnalyticsController; глобальный обработчик исключений RestExceptionHandler; типизированная иерархия исключений ApiExceptions
api.dto	Data Transfer Object для запросов и ответов REST API
workflow	Доменный сервис WorkflowService, репозитории CRUD-операций (WorkflowsRepository, WorkflowRevisionRepository, WorkflowVersionRepository, WorkflowSnapshotRepository), модели (GraphSkeleton, ConnectionSkeleton, GraphUpdatedEvent)
run	Ядро исполнения: WorkflowExecutionService (топологическая сортировка по алгоритму Кана + параллельное исполнение через CompletableFuture); RunEnqueueService; RunQueryService; репозитории WorkflowRunRepository, NodeRunRepository; конфигурация ExecutionConfig
executor	Интерфейс NodeExecutor, реестр NodeExecutorRegistry, общий движок песочницы ContainerSandboxRunner, предзагрузчик SandboxImageWarmer; реализации: HttpNodeExecutor, PythonNodeExecutor, JavaScriptNodeExecutor, DataflowNodeExecutors (filter/map/reduce/foreach/flatmap), TriggerWebhookExecutor, BranchSplitNodeExecutor + BranchSplitStrategies + SplitEnvelope, BranchMergeNodeExecutor
triggers	TriggerService, TriggerScheduler (cron на Spring TaskScheduler), SchedulerConfig, TriggersRepository
blob	BlobService (content-addressed-операции put/get), BlobIndexRepository — индекс хешей
ws	WorkflowWsController (входящие STOMP-сообщения), GraphBroadcastListener (преобразование доменных событий в WebSocket), WebSocketConfig

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Пакет	Назначение
analytics	AbAnalyticsService (агрегация по вариантам А/В-эксперимента), AbAnalyticsRepository (чтение variant-rows и excluded count), StatTest (z-тест двух пропорций, доверительные интервалы), AbAnalyticsDtos
seed	DemoWorkflowSeeder + DemoWorkflowPlan + подпакет seed.plans с готовыми планами демонстрационных workflow
config	Конфигурационные бины: OpenApiConfig, JacksonConfig, S3Config, S3Properties, WebMvcConfig
util	Утилиты: CanonicalJson (приведение JSON к канонической форме), Hashing (вычисление SHA-256)

Таблица 2. Модули серверной части и их назначение

Общий объём серверной части — около 55 файлов .kt суммарно порядка 3 500 строк исходного кода (без учёта тестов).

2.4. Ключевые классы

Класс	Назначение
WorkflowExecutionService	Ядро исполнения workflow: топологическая сортировка по алгоритму Кана, проверка ацикличности, параллельный запуск независимых узлов через CompletableFuture.allOf(...).thenApplyAsync(...) в пуле workflowExecutor (16 потоков), изоляция сбоев, трансляция событий
HttpNodeExecutor	Исполнитель HTTP-узла: запросы GET/POST/PUT/PATCH/DELETE, настраиваемые заголовки и тело, тайм-аут, обработка ошибочных ситуаций
ContainerSandboxRunner	Универсальный движок Docker-песочницы: формирование команды docker run со всеми флагами безопасности (--network none, --read-only, --memory 256m, --pids-limit 64, --cap-drop ALL и др.), управление временной директорией, передача stdin, тайм-аут, чтение stdout/stderr, разбор JSON-результата
PythonNodeExecutor / JavaScriptNodeExecutor	Специализированные исполнители Code Node для Python и JavaScript, делегирующие запуск контейнера общему

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Класс	Назначение
	ContainerSandboxRunner (базовые образы python:3.11-slim, node:20-alpine)
SandboxImageWarmer	Предзагрузка базовых образов песочницы на старте приложения для устранения cold-start
DataflowNodeExecutors	Пять реализаций операций потоков данных: filter, map, reduce, foreach, flatmap
BranchSplitNodeExecutor + BranchSplitStrategies + SplitEnvelope	Разделение входного потока на N вариантов по выбираемой стратегии (равные веса, заданные веса, round-robin) — основа А/В-эксперимента
BranchMergeNodeExecutor	Слияние результатов ветвей А/В-эксперимента с фиксацией выбранного варианта
AbAnalyticsService	Агрегация просмотров и конверсий по вариантам, расчёт доверительных интервалов, вызов статистического теста
StatTest	Z-тест двух пропорций для проверки статистической значимости разности конверсий
BlobService	Контент-адресуемое хранение в MinIO: канонизация JSON, вычисление SHA-256, идемпотентная запись, проверка целостности при чтении
TriggerScheduler	Регистрация cron-триггеров в Spring TaskScheduler; горячая перезагрузка расписания при изменении конфигурации триггеров
TriggerWebhookExecutor	Обработка входящих webhook-запросов: верификация токена, постановка запуска в очередь с переданным телом в качестве входа
GraphBroadcastListener	Подписчик на доменные события исполнения; публикация в STOMP-топик /topic/workflows/{id}/graph через SimpMessagingTemplate
WorkflowService	Доменный сервис управления workflow: создание, чтение, обновление с автоматическим созданием новой ревизии; создание именованных версий и снапшотов; восстановление к версии (append-only)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Класс	Назначение
CanonicalJson	Приведение JSON-дерева к канонической форме (сортировка ключей, нормализация пробелов и числовых значений) для детерминированного вычисления SHA-256
DemoWorkflowSeeder	Посев готовых демонстрационных workflow на старте приложения (определяются в seed/plans/)
RestExceptionHandler	Глобальный обработчик исключений (@RestControllerAdvice): преобразование исключений (ConstraintViolationException, NoSuchElementException, IllegalStateException и т.д.) в унифицированные REST-ответы

Таблица 3. Ключевые классы серверной части

2.5. Технологические зависимости

Технология	Версия	Назначение
Kotlin	2.2.21	Язык программирования
JDK	24	Runtime JVM
Spring Boot	3.5.3	Каркас приложения (Web, WebSocket, JDBC, Validation, Scheduler)
springdoc-openapi	2.7.0	Автодокументация REST API (OpenAPI 3.0 + Swagger UI)
Liquibase	4.x	Миграции схемы PostgreSQL
PostgreSQL JDBC	42.x	Драйвер для PostgreSQL 16
AWS SDK for Java (S3 client)	2.31.78	Клиент MinIO (S3-совместимый)
Jackson Kotlin module	актуальная	Сериализация JSON
JUnit Jupiter	5.x	Модульные тесты
Testcontainers	1.21.1	Интеграционные тесты с одноразовыми контейнерами
JaCoCo Maven Plugin	0.8.13	Отчёт покрытия по строкам и ветвям
Apache Maven	3.9	Сборка и управление зависимостями

Таблица 4. Технологические зависимости серверной части

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2.6. Инструкция по сборке

Предварительные требования: JDK 24, Maven 3.9+ (или встроенный Maven Wrapper `./mvnw`), Docker Engine 20.10+ (требуется для прогона интеграционных тестов через Testcontainers и для исполнения Code Node при ручной проверке).

Полная сборка с прогоном всех тестов и формированием отчёта покрытия:

```
cd backend && mvn clean verify
```

Сборка без тестов (для быстрых пересборок в development):

```
cd backend && mvn package -DskipTests
```

Результат сборки: исполняемый fat-JAR `backend/target/aelevena-{version}.jar` (обычно 50–80 МБ); HTML-отчёт покрытия в `backend/target/site/jacoco/index.html`; XML-отчёты по каждому тесту в `backend/target/surefire-reports/`.

Запуск собранного JAR:

```
java -jar backend/target/aelevena-{version}.jar
```

Альтернативный запуск в режиме разработки (с горячей перезагрузкой при изменениях):

```
cd backend && mvn spring-boot:run
```

2.7. Сборка Docker-образа

Сборка production Docker-образа выполняется через Dockerfile в корне `backend/`:

```
cd backend && docker build -t flux-backend:local .
```

Запуск в контейнере (с подключением к внешним PostgreSQL и MinIO):

```
docker run --rm -p 8080:8080 \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://host.docker.internal:5432/a11a \
  -e APP_S3_ENDPOINT=http://host.docker.internal:9000 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  flux-backend:local
```

Монтирование `/var/run/docker.sock` необходимо для возможности запускать sandbox-контейнеры Code Node (backend запускает их через `docker-cli`). В production-окружении вместо прямого монтирования сокета рекомендуется использовать `docker-socket-проxy` (включён в `deploy/docker-compose.prod.yml`).

2.8. Запуск тестов

Все автоматические тесты: `cd backend && mvn test` (только модульные) или `mvn verify` (модульные + интеграционные + отчёт покрытия).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.09.03-04 12 03-1

Запуск отдельного класса: `mvn test -Dtest=DataflowNodeExecutorsTest`.

Запуск одного метода: `mvn test -Dtest=DataflowNodeExecutorsTest#filterRetainsMatchingItems`.

Просмотр HTML-отчёта покрытия: открыть `backend/target/site/jacoco/index.html` в браузере.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (дата обращения: 05.12.2025).
2. Kotlin Documentation [Электронный ресурс]. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 08.12.2025).
3. Apache Maven Documentation [Электронный ресурс]. — URL: <https://maven.apache.org/guides/> (дата обращения: 14.01.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 12 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]